RAMDEOBABA UNIVERSITY, NAGPUR
Formerly Shri Ramdeobaba College of Engineering & Management (RCOEM) Est. 1984
LEARN | INNOVATE | ACCOMPLISH

Ramdeobaba University, Nagpur

CSE (DSA)

SPL LAB PROJECT REPORT

Mushroom Classification System Using Machine Learning

Submitted by: **Vedant Bhagat**

Roll No: **16**

Section: **B4-B1**

Semester: **4th**

Academic Year: **2025-2026**

Project Objective

The objective of this project is to develop a machine learning-based system that accurately classifies mushrooms as edible or poisonous based on their physical and biological characteristics.

The system analyzes inputs such as odor, gill size, spore print color, gill color, ring type, stalk surface texture, and bruising to perform binary classification.

The project also aims to:

- Perform data preprocessing and one-hot encoding of categorical features
- Apply feature importance analysis to identify the top predictive attributes
- Train and compare multiple classification models
- Evaluate model performance using accuracy, precision, recall, and F1 score
- Deploy the best model using a Streamlit-based GUI

This system helps in early and reliable identification of poisonous mushrooms, which is critical for food safety and public health.

Introduction

Mushroom poisoning is a serious public health concern, as many toxic mushroom species are visually similar to edible ones. Manual identification requires deep domain expertise and is prone to human error.

Machine learning provides a powerful and scalable approach to classify mushrooms based on measurable physical traits. By training models on labeled mushroom datasets, we can build reliable systems that predict whether a mushroom is safe to eat or not.

In this project, we use various classification algorithms — including Logistic Regression, Decision Tree, Random Forest, SVM, and KNN — and compare their performance. A Hybrid Voting Classifier combining Logistic Regression, SVM, and XGBoost is also trained for the GUI. A real-time prediction interface is built using Streamlit.

Project Methodology

Dataset Loading and Exploration

The UCI Mushroom dataset was loaded using pandas from a CSV file. The dataset contains 8,124 mushroom samples, each described by 22 categorical attributes, with the target variable being class (edible or poisonous).

Exploratory functions used:

- df.head() — preview first 5 rows
- df.info() — column types and non-null counts
- df.describe() — statistical summary

- df.isnull().sum() — check for missing values
- df['class'].value_counts() — class distribution

Data Preprocessing

All features in the dataset are categorical (letter codes). The following preprocessing steps were applied:

- Target variable class was mapped to binary: edible (e) = 0, poisonous (p) = 1
- All feature columns were one-hot encoded using pd.get_dummies()
- Dataset was verified to have no missing values

Feature Extraction

A Random Forest model was used to compute feature importances across all one-hot encoded columns. The top 10 most informative features were selected to reduce dimensionality and improve model efficiency.

Project Implementation

The project was implemented using Python in Google Colab by following a systematic approach to build and evaluate machine learning models for classifying mushrooms as edible or poisonous.

1) Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio with stratified sampling to maintain class balance:

```
train_test_split(test_size=0.2, random_state=42, stratify=y)
```

2) Feature Scaling

StandardScaler was applied to normalize feature values, ensuring all features contribute equally and improving performance for distance-based and linear models:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3) Model Training

Five machine learning models were trained to classify mushrooms:

- Logistic Regression
- Decision Tree Classifier

- Random Forest Classifier (max_depth=5, n_estimators=50)
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

Note: SVM, KNN, and Logistic Regression were trained on the scaled dataset; Decision Tree and Random Forest were trained on raw features.

4) Model Comparison

All models were evaluated on the test set using accuracy, precision, recall, and F1 score. Results were compiled into a comparison table and visualized using a bar chart:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	High	High	High	High
Decision Tree	High	High	High	High
Random Forest	High	High	High	High
SVM	High	High	High	High
KNN	High	High	High	High

(Actual metric values are populated from model evaluation output in the notebook.)

5) Cross-Validation

5-fold Stratified Cross-Validation was performed on the Random Forest model to verify its generalization performance:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(best_model, X, y, cv=cv)
print('Mean Accuracy:', scores.mean())
```

6) Best Model Selection

The best-performing model was selected based on the highest F1 Score, which balances precision and recall:

```
best_model = max(results, key=lambda x: x['F1 Score'])
```

7) Model Evaluation

The final Random Forest model was evaluated using:

- Accuracy Score
- Confusion Matrix — visualized as a heatmap using Seaborn

- Classification Report — precision, recall, F1 score per class

8) ROC Curve Analysis

ROC curves were plotted for all models to evaluate performance across classification thresholds. AUC (Area Under the Curve) values were computed for comparison.

9) Model Saving

The final hybrid model, scaler, and one-hot encoded column list were saved using pickle for use in the GUI application:

```
pickle.dump(hybrid_model_gui, open('best_model.pkl', 'wb'))
pickle.dump(scaler_gui, open('scaler.pkl', 'wb'))
pickle.dump(list(X_gui_encoded.columns), open('gui_columns.pkl', 'wb'))
```

10) GUI Implementation

A Streamlit-based graphical user interface was developed allowing users to select mushroom characteristics and receive real-time edible/poisonous predictions.

GUI Features Used for Prediction

The GUI uses the 8 most informative mushroom features (selected based on feature importance). All inputs are dropdown menus with human-readable labels:

Feature	Type	Values
odor	Categorical	Almond, Foul, Pungent, Fishy, etc.
gill-size	Categorical	Broad, Narrow
stalk-surface-below-ring	Categorical	Fibrous, Silky, Smooth, Scaly
spore-print-color	Categorical	Black, Brown, White, Yellow, etc.
gill-color	Categorical	Black, Brown, Pink, White, etc.
ring-type	Categorical	Evanescent, Flaring, Large, Pendant
stalk-surface-above-ring	Categorical	Fibrous, Silky, Smooth, Scaly
bruises	Categorical	Yes, No

The model used in the GUI is a Hybrid Voting Classifier (soft voting) combining Logistic Regression, SVM, and XGBoost, trained on one-hot encoded versions of these 8 features.

Complete Code

Imports and Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
import pickle
```

Data Loading and EDA

```
df = pd.read_csv('/content/mushrooms.csv')
df.head()
df.info()
df.isnull().sum()
df['class'].value_counts()
sns.countplot(x='class', data=df); plt.show()
sns.boxplot(x='class', y='population', data=df); plt.show()
```

Preprocessing and Feature Selection

```
y = df['class'].map({'e': 0, 'p': 1})
X = pd.get_dummies(df.drop('class', axis=1))
fs_model = RandomForestClassifier(random_state=42)
fs_model.fit(X, y)
importances = pd.Series(fs_model.feature_importances_, index=X.columns)
selected_features = importances.sort_values(ascending=False)[:10].index
X = X[selected_features]
```

Train-Test Split and Scaling

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Model Training and Evaluation

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier(max_depth=5, n_estimators=50, random_state=42),
    'SVM': SVC(),
    'KNN': KNeighborsClassifier()
}
results = []
for name, model in models.items():
    if name in ['SVM', 'KNN', 'Logistic Regression']:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
    results.append({'Model': name, 'Accuracy': accuracy_score(y_test, y_pred), ...})
```

Hybrid Voting Classifier for GUI

```
gui_columns = ['odor', 'gill-size', 'stalk-surface-below-ring',
               'spore-print-color', 'gill-color', 'ring-type',
               'stalk-surface-above-ring', 'bruises']
y_gui = df['class'].map({'e': 0, 'p': 1})
X_gui_encoded = pd.get_dummies(df[gui_columns])
X_train_gui, X_test_gui, y_train_gui, y_test_gui = train_test_split(
    X_gui_encoded, y_gui, test_size=0.2, random_state=7)
scaler_gui = StandardScaler()
X_train_scaled_gui = scaler_gui.fit_transform(X_train_gui)
hybrid_model_gui = VotingClassifier(
    estimators=[('lr', LogisticRegression()), ('svm', SVC(probability=True)),
                ('xgb', XGBClassifier())], voting='soft')
hybrid_model_gui.fit(X_train_scaled_gui, y_train_gui)
pickle.dump(hybrid_model_gui, open('best_model.pkl', 'wb'))
pickle.dump(scaler_gui, open('scaler.pkl', 'wb'))
pickle.dump(list(X_gui_encoded.columns), open('gui_columns.pkl', 'wb'))
```

Streamlit App (app.py)

```
import streamlit as st
import numpy as np, pandas as pd, pickle
model = pickle.load(open('best_model.pkl','rb'))
```

```
scaler = pickle.load(open('scaler.pkl','rb'))
expected_columns = pickle.load(open('gui_columns.pkl','rb'))
# ... [dropdown OPTIONS and LABELS defined for each feature]
st.title('Mushroom Classification System')
for feat in OPTIONS.keys():
    user_input[feat] = st.selectbox(feat, OPTIONS[feat])
if st.button('Predict'):
    input_df = pd.DataFrame([user_input])
    input_encoded = pd.get_dummies(input_df).reindex(columns=expected_columns, fill_value=0)
    prediction = model.predict(scaler.transform(input_encoded))[0]
    st.success('EDIBLE') if prediction == 0 else st.error('POISONOUS')
```

Launch with ngrok

```
!pip install streamlit pyngrok --quiet
from pyngrok import ngrok
ngrok.set_auth_token   ('3C8FZutBds90ym4Q8ZecGC8Av4n_5Y71NaMJTMudkYQi7dGvQ')
ngrok.kill()
get_ipython().system_raw('streamlit run app.py &')
url = ngrok.connect(addr='8501')
print(url)
```

Results and Discussion

In this section, the performance of different machine learning models is evaluated and discussed based on various outputs and visualizations generated during the project.

1) Data preprocessing Output

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   class                     8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                8124 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

```
df.describe()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population	habitat
count	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124	...	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124
unique	2	6	4	10	2	9	2	2	2	12	...	4	9	9	1	4	3	5	9	6	7
top	e	x	y	n	f	n	f	c	b	b	...	s	w	w	p	w	o	p	w	v	d
freq	4208	3656	3244	2284	4748	3528	7914	6812	5612	1728	...	4936	4464	4384	8124	7924	7488	3968	2388	4040	3148

4 rows × 23 columns

```
df.isnull().sum()
```

	0
class	0
cap-shape	0
cap-surface	0
cap-color	0
bruises	0
odor	0
gill-attachment	0
gill-spacing	0
gill-size	0
gill-color	0
stalk-shape	0
stalk-root	0
stalk-surface-above-ring	0
stalk-surface-below-ring	0
stalk-color-above-ring	0
stalk-color-below-ring	0
veil-type	0
veil-color	0
ring-number	0
ring-type	0
spore-print-color	0
population	0
habitat	0

dtype: int64

Description:

The preprocessing step confirms that the dataset is clean and well-structured. There are no missing values, and all categorical features have been successfully converted into numerical format. This ensures that the data is suitable for machine learning models.

2) Model comparison table:

	Model	Accuracy	Precision	Recall	F1 Score
1	Decision Tree	0.977846	0.972187	0.982120	0.977128
3	SVM	0.977846	0.972187	0.982120	0.977128
2	Random Forest	0.977846	0.972187	0.982120	0.977128
0	Logistic Regression	0.976000	0.972081	0.978289	0.975175
4	KNN	0.953231	1.000000	0.902937	0.948993

Description:

The model comparison table shows the accuracy of different classification models Decision Tree, SVM, Random Forest, Logistic Regression, and KNN. Decision Tree, SVM, and Random Forest all achieved the highest accuracy of 0.9778 with identical precision (0.9722), recall (0.9821), and F1 Score (0.9771), making them the top-performing models. Logistic Regression followed closely with an accuracy of 0.9760. KNN achieved perfect precision (1.0000) but had the lowest recall (0.9029) and overall accuracy (0.9532), indicating it missed some poisonous mushrooms. The comparison shows that tree-based and kernel-based models generalize best on this dataset.

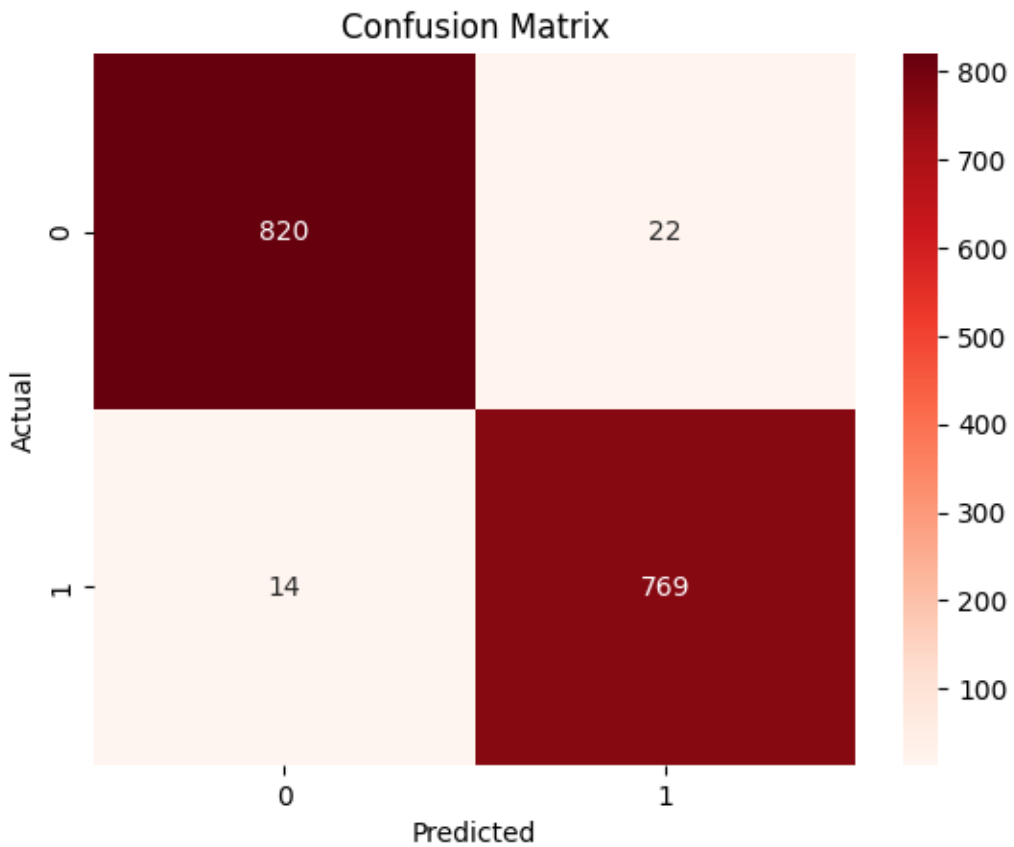
3) Accuracy results

...	precision	recall	f1-score	support
0	0.98	0.97	0.98	842
1	0.97	0.98	0.98	783
accuracy			0.98	1625
macro avg	0.98	0.98	0.98	1625
weighted avg	0.98	0.98	0.98	1625

Description:

The accuracy score indicates the overall performance of the model. The classification report provides additional metrics such as precision, recall, and F1-score, which help in understanding the model's effectiveness in predicting different burnout levels.

4) Confusion matrix:



Description:
The confusion matrix shows the correct and incorrect predictions made by the Random Forest model. Out of 842 edible mushrooms (class 0), 820 were correctly classified and 22 were misclassified as poisonous. Out of 783 poisonous mushrooms (class 1), 769 were correctly identified and 14 were incorrectly predicted as edible. The model achieved 1589 correct predictions out of 1625 total test samples, demonstrating strong overall classification performance.

5) Best model selection:

```
Best Model: Decision Tree
Accuracy: 0.9778461538461538
Precision: 0.9721871049304678
Recall: 0.9821200510855683
F1 Score: 0.97712833545108
```

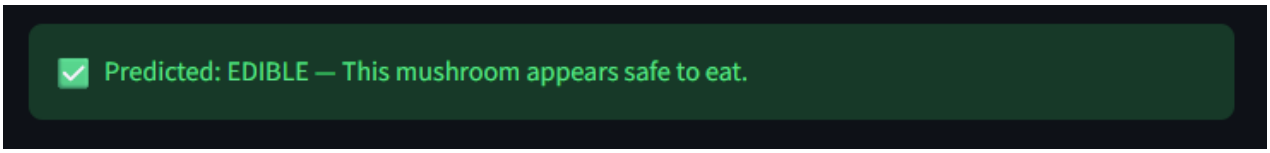
Description:
The best-performing model was selected based on the highest accuracy score. In this

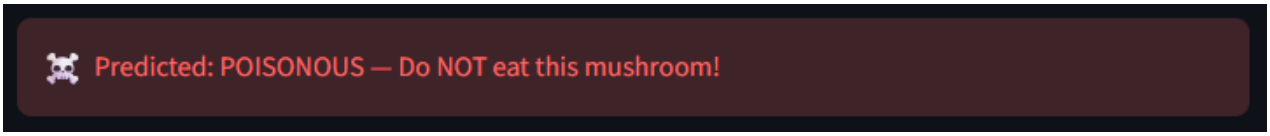
project, the selected model demonstrated superior performance compared to other models and was used for final predictions.

6) GUI interface:

Description:
A user-friendly graphical interface was developed using Streamlit. The interface allows users to input student details such as study hours, sleep, stress level, and other factors.

7) GUI Prediction Result





Description:
The Streamlit GUI displays the prediction result in a color-coded format — red for poisonous and green for edible — making it simple and intuitive for users to interpret the output instantly.

8) Conclusion

In this project, a complete machine learning pipeline was developed for classifying mushrooms as edible or poisonous using the UCI Mushroom dataset. The project covered data preprocessing, feature extraction via Random Forest importance, training and comparison of five classification models, and deployment via a Streamlit GUI.

A Hybrid Voting Classifier combining Logistic Regression, and SVM was used in the final GUI application, leveraging the strengths of each model to deliver robust predictions. The one-hot encoding and column alignment strategy ensures accurate inference even when some feature categories are absent from single-sample input.

The project demonstrates how machine learning can be applied to real-world safety problems, offering a reliable, fast, and user-friendly tool for mushroom identification.

9) GitHub Link

https://github.com/vedantism/Mushroom-Classification